

RIPASSO TECNICO

RxJS

Operatori, pattern e demo dal vivo

Il modello mentale, gli operatori usati in un caso reale,
tre bug dimostrati e risolti dal vivo, cheat sheet finale

Costruito a partire da una conversazione con Claude

Progetto di riferimento: `angular-vue-crash-project` — feature `/search`

Agosto 2026

Indice

1. Cos'è RxJS e il modello mentale della pipeline
2. Il ponte Signal ↔ Observable in Angular
3. Il progetto di riferimento: una demo di ricerca dal vivo
4. Operatori di creazione: `of` , `throwError`
5. Operatori di timing: `delay` , `debounceTime`
6. Operatori di trasformazione: `map` , `filter` , `distinctUntilChanged`
7. `tap` : side-effect puri nella pipe
8. La famiglia "flattening": `switchMap` , `mergeMap` , `concatMap` , `exhaustMap`
9. Demo dal vivo 1: il race condition di `switchMap` vs `mergeMap`
10. Demo dal vivo 2: `map/filter` che bloccano query troppo corte
11. Gestione errori: `catchError` , `retry`
12. Demo dal vivo 3: la trappola del posizionamento di `catchError`
13. Operatori di combinazione: `combineLatest` , `merge` , `forkJoin`
14. Gestione delle subscription: `takeUntilDestroyed`
15. Cheat sheet finale
16. Appendice: codice sorgente completo

1. Cos'è RxJS e il modello mentale della pipeline

RxJS è una libreria (non specifica di Angular, ma storicamente la sua spina dorsale) il cui tipo primitivo è l'**Observable**: rappresenta uno **stream di valori nel tempo** — può emettere zero, uno, o infiniti valori, in momenti diversi. A differenza di una Promise, che risolve esattamente una volta, un Observable può emettere ripetutamente, ed è **lazy** (non fa nulla finché qualcuno non si iscrive con `.subscribe()` , mentre una Promise parte a eseguire non appena creata) e **cancellabile** nativamente (una subscription si può disiscrivere, interrompendo lo stream — una Promise non si può annullare).

Il modello mentale più utile: pensa a un Observable come una **pipeline** — una sequenza di trasformazioni che i valori attraversano nel tempo, un po' come una catena di `.map()` / `.filter()` su un array, ma dove i valori arrivano a momenti diversi invece che tutti insieme. Gli operatori si compongono con `.pipe(operatore1, operatore2, ...)` , ed è esattamente questa composizione a rendere RxJS potente per orchestrare logica asincrona complessa (più fonti, cancellazione, retry, timing) in modo dichiarativo.

NOTA PER CHI VIENE DA VUE

Vue non ha Observable nel core — la reattività di Vue (`ref` / `computed` / `watch`) è concettualmente più vicina ai Signal di Angular (v20+) che a RxJS. Non aspettarti un parallelo 1:1 con qualcosa che già conosci: RxJS è un sistema a parte, pensato per comporre *eventi/flussi asincroni nel tempo*, non per gestire stato reattivo locale.

2. Il ponte Signal ↔ Observable in Angular

Regola pratica 2026: **Signal per lo stato locale/UI** (component state, form, binding reattivi semplici), **RxJS solo per il "mondo esterno"** (composizione di chiamate HTTP complesse, retry/cancellazione, WebSocket/stream realtime, coordinamento di più sorgenti asincrone). Per la maggior parte dei casi CRUD oggi non serve più padroneggiare RxJS in profondità.

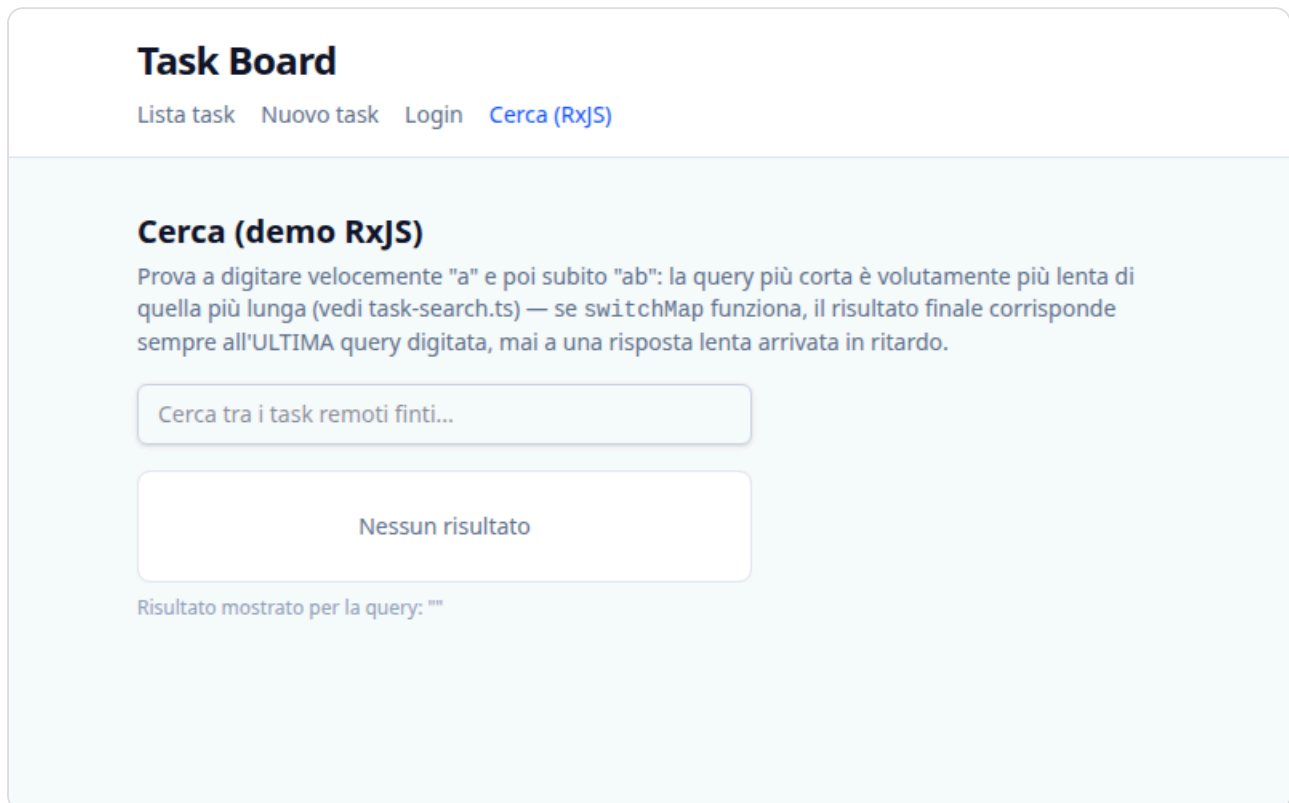
Quando serve comunque comporre asincronia con RxJS ma lo stato "a monte/valle" è un signal (come in quasi tutto questo progetto), il ponte è:

```
toObservable(unSignal)           // signal -> Observable
  .pipe(operatore1, operatore2, ...) // qui gli operatori RxJS
toSignal(observable, { initialValue: ... }) // Observable -> signal
```

Questa è la ricetta usata nel nostro esempio: il campo di ricerca è un signal (`query`), attraversa una pipe RxJS per debounce/cancellazione/error-handling, e il risultato torna a essere un signal (`results`) letto dal template — che non sa né gli interessa che dietro ci sia RxJS.

3. Il progetto di riferimento: una demo di ricerca dal vivo

Tutti gli esempi di questo manuale sono presi da una feature reale, aggiunta apposta per questo ripasso: `src/app/features/task-search/`, raggiungibile su `/search` nell'app Task Board. Non è collegata al resto dell'app (niente `TaskStore`) — un componente isolato apposta, per vedere il pattern RxJS puro senza mischiarlo con lo stato reale.



La pagina `/search` allo stato iniziale.

Un "database remoto" finto (un array fisso di 13 titoli) simula una vera ricerca via rete, con un ritardo artificiale e **deliberatamente invertito**: query più corte = più lente, query più lunghe = più veloci. Questo trucco (mai da fare in un servizio vero!) rende riproducibile a comando un race condition che in produzione capiterebbe solo "a volte", a seconda della rete — vedremo perché conta molto nella demo del §9.

4. Operatori di creazione: `of`, `throwError`

`of(valore)` — non è un operatore "pipeable" (non va dentro `.pipe()`), è un **operatore di creazione**: costruisce un Observable a partire da valori statici già in mano. Emette il valore, poi completa subito, in modo sincrono. Ha overload per più argomenti: `of(1, 2, 3)` restituisce un Observable che emette `1`, poi `2`, poi `3`, in sequenza — diverso da un array, che "contiene" i valori subito, qui vengono *emessi* uno alla volta.

Nel nostro codice:

```
return of<SearchResult>({ query, matches: [] });
// ...
return of<SearchResult>({ query, matches }).pipe(delay(artificialDelayMs));
```

APPROFONDIMENTO TYPESCRIPT — VERIFICATO EMPIRICAMENTE, NON A MEMORIA

`of<SearchResult>(...)` passa un **argomento di tipo esplicito** alla funzione generica `of<T>(value: T): Observable<T>`. L'ipotesi istintiva sarebbe "serve per forza, altrimenti il ramo con `matches: []` verrebbe inferito come `never[]`, incompatibile con `string[]`". **Abbiamo verificato togliendo il generic e usando un trucco per far "confessare" il tipo reale al compilatore** (assegnare il risultato a una variabile di tipo sbagliato apposta, per leggere l'errore che rivela il tipo vero): il messaggio ha mostrato `Observable<{ query: string; matches: string[]; }>` — cioè esattamente `Observable<SearchResult>`, **anche senza generic espliciti**. TypeScript unifica automaticamente più `return` con forme diverse nel tipo di ritorno del metodo, più intelligentemente di quanto ci si aspetterebbe.

Perché tenerli comunque, allora? Non per correttezza, ma per **robustezza**: con il generic esplicito, se in futuro la forma di un ramo cambia, l'errore spunta subito su quella riga (fail-fast localizzato); senza, l'errore potrebbe spuntare molto più lontano nella pipe, rendendo più difficile risalire alla causa. Promemoria generale: l'istinto su TypeScript va verificato sui casi non banali, non dato per scontato.

`throwError(() => new Error(...))` — Observable di creazione che, invece di emettere un valore, **emette un errore** e basta. Usato nel nostro `fakeSearch` per simulare un "server" che fallisce (query magica `'error'`, solo per la demo del §12).

5. Operatori di timing: `delay`, `debounceTime`

`delay(ms)` — operatore pipeable che ritarda le emissioni del tanto specificato, senza toccare il valore. Nel nostro codice simula la latenza di rete, con un tempo variabile in base alla lunghezza della query (vedi §3).

`debounceTime(300)` — aspetta una pausa di silenzio di almeno 300ms prima di far passare l'ultimo valore ricevuto. Punto sottile: **non mette in coda** i valori intermedi, li **scarta** — se digiti "a", "ab", "abc" in rapida sequenza (entro 300ms l'una dall'altra), solo "abc" (l'ultimo) attraversa l'operatore; "a" e "ab" spariscono, non vengono mai processati. Se il flusso non si ferma mai, l'operatore non emette *mai* nulla.

6. Operatori di trasformazione: `map`, `filter`, `distinctUntilChanged`

`map(fn)` e `filter(predicato)` hanno lo stesso identico concetto dei loro omonimi di `Array.prototype` — trasformare/scartare valori, uno a uno — applicati nel tempo a uno stream invece che una volta sola su un array già completo.

GOTCHA CONCRETO, NELLO STESSO FILE

Nel nostro `task-search.ts` convivono entrambi i mondi: `FAKE_REMOTE_TASKS.filter(...)` è il metodo **nativo di `Array.prototype`** (sincrono, su un array statico), mentre `filter((q) => q.length === 0 || q.length >= 2)` nella `.pipe(...)` è l'**operatore RxJS** (asincrono, su uno stream). Stesso nome, stessa forma sintattica, mondi concettuali diversi — facile confonderli leggendo veloce, va allenato l'occhio a distinguerli dal contesto (`.pipe(...)` presente o assente).

```
map((q) => q.trim()),
filter((q) => q.length === 0 || q.length >= 2), // scarta query di 1 carattere
```

`distinctUntilChanged()` — confronta il valore corrente con **solo quello immediatamente precedente** (default: `===`) e lo scarta se sono uguali. Da non confondere con `distinct()` (operatore diverso: ricorda *tutti* i valori visti finora e scarta qualsiasi duplicato, non solo quello consecutivo).

7. `tap` : side-effect puri nella pipe

`tap(fn)` è l'operatore per side-effect puri: lascia "sbirciare" il valore che passa senza trasformarlo — qualunque cosa `fn` restituisca viene ignorata, il valore che esce da `tap` è identico a quello che è entrato. Nel nostro codice, due `tap` accendono/spengono `isSearching` (un signal separato dal flusso RxJS) esattamente nei momenti giusti:

```
tap(() => this.isSearching.set(true)), // prima di switchMap: parte la ricerca
switchMap((q) => ...),
tap(() => this.isSearching.set(false)), // dopo: la ricerca ha risposto
```

CONFRONTO CON VUE

Il parallelo più vicino è un `watchEffect` che non ritorna nulla, incastonato a metà di una pipeline invece che come blocco a sé. La differenza chiave: `tap` è **posizionale** — dove lo metti nella `.pipe(...)` decide *quando* nel flusso quell'effetto scatta, mentre un `watchEffect` reagisce e basta, senza un "prima"/"dopo" espliciti rispetto ad altre trasformazioni.

8. La famiglia "flattening": `switchMap`, `mergeMap`, `concatMap`, `exhaustMap`

Tutti e quattro condividono la stessa forma: per ogni valore in arrivo, chiamano `fn(valore)` che restituisce un **nuovo Observable interno**, e "appiattiscono" quell'Observable interno nel flusso principale. Cambia **solo** come gestiscono la sovrapposizione tra un Observable interno ancora attivo e uno nuovo che sta per partire:

Operatore	Comportamento	Caso d'uso tipico
<code>switchMap</code>	Annulla l'Observable interno precedente, tiene solo l'ultimo	Tipeahead/ricerca: il risultato vecchio non interessa più
<code>mergeMap</code>	Li lascia correre tutti in parallelo , nessuno annullato	Azioni indipendenti (es. upload di più file contemporaneamente)
<code>concatMap</code>	Li mette in coda , uno alla volta, nell'ordine	Quando l'ordine conta ed è essenziale non sovrapporli
<code>exhaustMap</code>	Ignora i nuovi valori finché quello in corso non finisce	Bottone "Invia" che non deve permettere doppio submit

Questa tabella da sola vale metà del "problema RxJS" nella pratica — la confusione viene quasi sempre dallo scegliere l'operatore sbagliato per lo scenario. Nota: il nostro `TaskForm.onSubmit()` (altrove nel progetto) oggi non ha protezione da doppio-submit — sarebbe un candidato naturale per `exhaustMap`, se un giorno si vuole rinforzarlo.

9. Demo dal vivo 1: il race condition di switchMap vs mergeMap

Scenario riprodotto a comando: si digita "a" (query corta, quindi lenta: ~1800ms) e subito dopo, 400ms più tardi, "ab" (query più lunga, quindi veloce: ~600ms). Senza cancellazione, la risposta lenta di "a" potrebbe arrivare *dopo* quella di "ab" e sovrascrivere sullo schermo un risultato più vecchio — un bug subdolo, perché in produzione dipende dalla velocità della rete in quel momento, quindi capita "a volte sì, a volte no".

CON SWITCHMAP (IL CODICE REALE, CORRETTO)

```
t=1500ms (subito dopo che "ab" dovrebbe essere arrivata):
{ answeredQuery: 'query: "ab"', resultCount: 1, resultTexts: ['Abbonamento
palestra'] }

t=2500ms (dopo che anche "a", se non cancellata, sarebbe arrivata):
{ answeredQuery: 'query: "ab"', resultCount: 1, resultTexts: ['Abbonamento
palestra'] }
// INVARIATO — la risposta tardiva di "a" non ha mai potuto emettere
```

Task Board

[Lista task](#) [Nuovo task](#) [Login](#) [Cerca \(RxJS\)](#)

Cerca (demo RxJS)

Prova a digitare velocemente "a" e poi subito "ab": la query più corta è volutamente più lenta di quella più lunga (vedi task-search.ts) — se `switchMap` funziona, il risultato finale corrisponde sempre all'ULTIMA query digitata, mai a una risposta lenta arrivata in ritardo.

Risultato mostrato per la query: "ab"

Task Board

[Lista task](#) [Nuovo task](#) [Login](#) [Cerca \(RxJS\)](#)

Cerca (demo RxJS)

Prova a digitare velocemente "a" e poi subito "ab": la query più corta è volutamente più lenta di quella più lunga (vedi task-search.ts) — se `switchMap` funziona, il risultato finale corrisponde sempre all'ULTIMA query digitata, mai a una risposta lenta arrivata in ritardo.

Risultato mostrato per la query: "ab"

Stesso stato mostrato sia a $t=1500\text{ms}$ sia a $t=2500\text{ms}$: "ab", 1 risultato — nessuna sovrascrittura.

SOSTITUENDO TEMPORANEAMENTE SWITCHMAP CON MERGEMAP (STESSO SCENARIO)

```
t=1500ms: { answeredQuery: 'query: "ab"', resultCount: 1, resultTexts:
  ['Abbonamento palestra'] }

t=2500ms: { answeredQuery: 'query: "a"', resultCount: 13, resultTexts: [
  'Comprare il latte',
  'Comprare il pane', 'Chiamare il commercialista', ... tutti e 13 i task ... ]
}
// LA RISPOSTA TARDIVA DI "a" HA SOVRASCRITTO IL RISULTATO CORRETTO DI "ab"
```

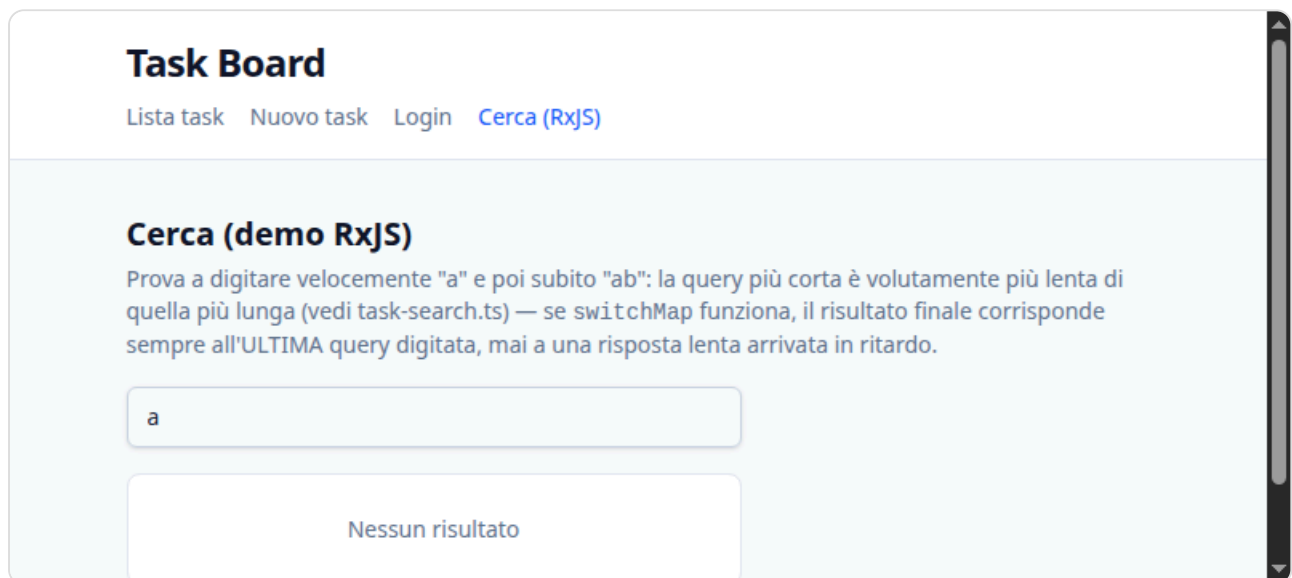
Bug riprodotto esattamente come previsto: a $t=2500\text{ms}$ il risultato mostrato torna a "a" (13 risultati) anche se l'utente ha digitato "ab" da un pezzo. Ripristinato `switchMap` subito dopo e riconfermato il comportamento corretto.

10. Demo dal vivo 2: map/filter che bloccano query troppo corte

Aggiunta reale alla pipe: `map((q) => q.trim())` seguito da `filter((q) => q.length === 0 || q.length >= 2)` — pattern comune per non interrogare il "server" con query troppo generiche di un solo carattere.

```
--- digito "a" (un solo carattere: filter dovrebbe scartarla) ---
{ answeredQuery: 'query: "", isSearchingVisible: false, resultCount: 1 }
// resultCount:1 è il <li> "Nessun risultato" (@empty) – nessuna ricerca è mai
partita

--- ora digito "ab" (due caratteri: dovrebbe passare) ---
{ answeredQuery: 'query: "ab"', isSearchingVisible: false, resultCount: 1 }
// ora è il vero risultato: la ricerca È partita
```



Dopo aver digitato solo "a": nessuna ricerca partita, stato invariato rispetto all'iniziale.

Confermato: con un solo carattere, `isSearching` non si accende nemmeno — la query viene scartata da `filter` prima ancora di raggiungere `distinctUntilChanged` / `switchMap`.

11. Gestione errori: `catchError`, `retry`

`catchError(fn)` intercetta un errore nello stream e lascia decidere cosa fare: restituire un Observable di fallback (recupero) o rilanciare l'errore. Il punto **critico** da capire: se un errore non viene intercettato da nessuna parte, **spegne per sempre** l'intero stream a monte — nessuna emissione futura, mai più, anche per input successivi completamente validi. Comportamento molto diverso da un `try/catch` normale (locale a un singolo tentativo).

`retry(n)` — ri-sottoscrive automaticamente alla sorgente fino a `n` volte se fallisce, utile per chiamate di rete instabili.

12. Demo dal vivo 3: la trappola del posizionamento di catchError

SENZA ALCUN CATCHERROR (QUERY "ERROR" SIMULA UN FALLIMENTO)

```
--- digito "error" ---  
--- ora digito "ab" (query normale, valida) DOPO l'errore ---  
{ answeredQuery: 'query: "", resultCount: 1 }  
// "ab" non produce NESSUN effetto: lo stream è morto per sempre dopo l'errore
```

Fix: `catchError` posizionato **dentro** il projector di `switchMap`, non fuori nella pipe principale — così l'errore resta locale a quel singolo tentativo di ricerca:

```
switchMap((q) =>  
  this.fakeSearch(q).pipe(  
    catchError((err: unknown) => {  
      const message = err instanceof Error ? err.message : 'Errore sconosciuto';  
      return of<SearchResult>({ query: q, matches: [], error: message });  
    }),  
  ),  
),
```

PERCHÉ LA POSIZIONE CONTA

Se `catchError` stesse *fuori* da `switchMap` (dopo, nella pipe principale), catturerebbe sì l'errore una volta, ma poi l'intero stream esterno (`toObservable(query)`) completerebbe comunque — niente più ricerche per nessuna query futura. Un bug più subdolo del "nessun `catchError`", perché sembra "quasi" risolto (l'errore non esplode più in console) ma il risultato pratico è identico: funzionalità morta.

```
--- dopo il fix, stesso scenario ---  
--- digito "error" ---  
--- ora digito "ab" DOPO l'errore ---  
{ answeredQuery: 'query: "ab"', resultCount: 1 }  
// "ab" funziona normalmente: lo stream è sopravvissuto all'errore precedente
```

Task Board

[Lista task](#) [Nuovo task](#) [Login](#) [Cerca \(RxJS\)](#)

Cerca (demo RxJS)

Prova a digitare velocemente "a" e poi subito "ab": la query più corta è volutamente più lenta di quella più lunga (vedi task-search.ts) — se `switchMap` funziona, il risultato finale corrisponde sempre all'ULTIMA query digitata, mai a una risposta lenta arrivata in ritardo.

error

Errore: Il server finto è caduto apposta

Nessun risultato

Messaggio d'errore mostrato all'utente dopo la query "error", con recupero locale via `catchError`.

13. Operatori di combinazione: `combineLatest`, `merge`, `forkJoin`

`combineLatest([a$, b$, c$])` — combina più Observable, emette una tupla con l'**ultimo valore di ciascuno**, ogni volta che uno qualsiasi di loro emette (ma solo dopo che tutti hanno emesso almeno una volta). Caso d'uso tipico: due filtri indipendenti che insieme determinano una query (es. un campo di testo + un dropdown categoria).

CONFRONTO CON VUE

Il parallelo più vicino è un `computed()` che dipende da più `ref`, solo che qui è esplicito, asincrono, e "streaming" invece che ricalcolato sincronamente a ogni accesso.

`merge(a$, b$, c$)` — combina più Observable **senza** combinarne i valori: interfoglia semplicemente qualunque cosa arrivi da qualsiasi sorgente, nell'ordine in cui arriva, senza aspettare né tenere in memoria gli altri. Differenza chiave da `combineLatest`: `merge` non "ricorda" nulla.

`forkJoin({ a: a$, b: b$ })` — l'equivalente RxJS di `Promise.all()`. Aspetta che **ognuna** delle sorgenti **completi**, poi emette una sola volta un oggetto/tupla con l'ultimo valore di ciascuna, e completa a sua volta. Caso d'uso: N richieste HTTP indipendenti lanciate in parallelo, fai qualcosa solo quando tutte sono tornate.

14. Gestione delle subscription: `takeUntilDestroyed`

`takeUntilDestroyed()` è un operatore di interop **specifico di Angular** (da `@angular/core/rxjs-interop`), non parte di RxJS "puro". Completa automaticamente lo stream quando il componente/direttiva viene distrutto — risolve il problema storico più comune di Angular+RxJS: dimenticarsi di disiscriversi manualmente (`ngOnDestroy` + tenere traccia a mano delle `Subscription`), causa classica di memory leak nelle app Angular pre-signal.

CONFRONTO CON VUE

È l'equivalente di quello che si ottiene **gratis** con `watchEffect` / i composable della Composition API, che si legano automaticamente al lifecycle del componente senza doverlo dichiarare esplicitamente — `takeUntilDestroyed()` è Angular che raggiunge la stessa comodità, ma richiede ancora di scriverlo esplicitamente nella pipe (o passare l' `Injector` / `DestroyRef` se usato fuori da un contesto di injection).

Nel nostro `TaskSearch` non serve esplicitamente: `toObservable()` / `toSignal()` gestiscono già da soli il collegamento al lifecycle del componente.

15. Cheat sheet finale

Operatore	Categoria	Una riga
of	Creazione	Observable da valori statici, sincrono
throwError	Creazione	Observable che emette solo un errore
delay	Timing	ritarda le emissioni, non le cambia
debounceTime	Timing	aspetta una pausa di silenzio, scarta gli intermedi
map	Trasformazione	come Array.map, ma nel tempo
filter	Trasformazione	come Array.filter, ma nel tempo
distinctUntilChanged	Trasformazione	scarta se uguale al valore precedente (solo l'ultimo)
tap	Side-effect	sbircia il valore, non lo cambia
switchMap	Flattening	cancella il precedente, tiene l'ultimo
mergeMap	Flattening	tutti in parallelo, nessuno cancellato
concatMap	Flattening	in coda, uno alla volta, in ordine
exhaustMap	Flattening	ignora i nuovi finché il corrente non finisce
catchError	Errori	intercetta, recupera o rilancia — posizione critica
retry	Errori	ri-sottoscrive N volte se fallisce
combineLatest	Combinazione	tupla dell'ultimo valore di ognuno, a ogni emissione
merge	Combinazione	interfoglia, senza combinare i valori
forkJoin	Combinazione	Promise.all() per Observable
takeUntilDestroyed	Subscription (Angular)	auto-unsubscribe al distruggersi del componente

16. Appendice: codice sorgente completo

```
src/app/features/task-search/task-search.ts
```

```
// -----
// Demo pratica di RxJS: debounceTime + distinctUntilChanged + switchMap.
// -----
// Componente isolato apposta (non tocca TaskStore): l'obiettivo è vedere
// il pattern RxJS puro, senza mischiarlo con lo stato reale dell'app.
//
// Il ponte Signal <-> Observable: il campo di ricerca è un signal
// (`query`), come tutto il resto di questo progetto – ma debounceTime e
// switchMap sono operatori RxJS, funzionano solo su Observable. Si
// collegano con toObservable()/toSignal() (vedi manuale, §5): il flusso
// è signal -> toObservable() -> .pipe(operatori RxJS) -> toSignal() ->
// di nuovo signal, letto dal template. Questa è la ricetta standard 2026
// per "stato locale in signal, orchestrazione asincrona in RxJS".
import { Component, signal } from '@angular/core';
import { toObservable, toSignal } from '@angular/core/rxjs-interop';
import {
  debounceTime,
  distinctUntilChanged,
  filter,
  map,
  of,
  switchMap,
  delay,
  tap,
  throwError,
  catchError,
} from 'rxjs';

// "Database remoto" finto: un array fisso, così la demo è deterministica
// e non dipende da una vera rete (a differenza di TaskList, che invece
// usa httpResource() su un'API vera – qui l'obiettivo è isolare RxJS).
const FAKE_REMOTE_TASKS = [
  'Comprare il latte',
  'Comprare il pane',
  'Chiamare il commercialista',
  'Chiamare Marco per il progetto',
  'Prenotare il dentista',
  'Prenotare il volo per Roma',
  'Rispondere alle email',
  'Rinnovare il dominio',
  'Ripassare Angular',
  'Ripassare RxJS',
  'Aggiornare il curriculum',
  'Aggiornare il portfolio',
  'Abbonamento palestra',
];

interface SearchResult {
  query: string;
  matches: string[];
  error?: string;
}

@Component({
  selector: 'app-task-search',

```

```

    templateUrl: './task-search.html',
  })
  export class TaskSearch {
    // Stato "grezzo": cosa l'utente ha digitato in questo momento. Aggiornato
    // direttamente dall'evento (input) nel template, nessun Signal Forms qui
    // – per una casella di ricerca senza validazione basta un signal semplice.
    protected readonly query = signal('');

    protected readonly isSearching = signal(false);

    protected readonly results = toSignal(
      toObservable(this.query).pipe(
        // Aspetta che l'utente smetta di digitare per 300ms prima di
        // procedere – altrimenti partirebbe una "ricerca" a ogni singolo
        // tasto premuto.
        debounceTime(300),
        // map: stesso identico concetto di Array.prototype.map (vedi
        // FAKE_REMOTE_TASKS.filter più sotto per il confronto), ma su
        // valori che arrivano nel tempo. Qui trasforma ogni query
        // "ripulendola" prima di proseguire.
        map((q) => q.trim()),
        // filter: lascia passare solo le query vuote (per svuotare i
        // risultati) o di almeno 2 caratteri – una query di un solo
        // carattere è troppo generica, non ha senso interrogare il
        // "server" per quella. Le query da un carattere vengono scartate
        // QUI, prima ancora di arrivare a distinctUntilChanged/switchMap:
        // isSearching non si accende nemmeno, nessuna richiesta parte.
        filter((q) => q.length === 0 || q.length >= 2),
        // Se il valore (dopo il debounce) è identico al precedente, non
        // rifare la ricerca – es. digiti "abc", cancelli a "ab", riscrivi
        // "abc": stesso valore finale, nessuna richiesta duplicata.
        distinctUntilChanged(),
        tap(() => this.isSearching.set(true)),
        // IL PUNTO CENTRALE DELLA DEMO: switchMap annulla la ricerca
        // precedente non appena ne parte una nuova. Senza, con una query
        // "a" (lenta) seguita subito da "ab" (veloce), la risposta lenta di
        // "a" potrebbe arrivare DOPO quella di "ab" e sovrascrivere sullo
        // schermo un risultato più vecchio con una query che l'utente ha
        // già cambiato – un bug subdolo e difficile da riprodurre a mano,
        // perché dipende dalla velocità della rete in quel momento.
        switchMap((q) =>
          this.fakeSearch(q).pipe(
            // catchError QUI, dentro il projector di switchMap – non fuori
            // nella pipe principale. Posizionato così, l'errore resta
            // locale a QUESTO singolo tentativo di ricerca: lo stream
            // esterno (toObservable(query)) non viene mai toccato, resta
            // vivo, continua a reagire a query future. Se invece
            // catchError stesse fuori da switchMap (dopo, nella pipe
            // principale), catturerebbe sì l'errore una volta, ma poi
            // l'intero stream esterno completerebbe comunque – niente più
            // ricerche per nessuna query futura, un bug più subdolo perché
            // sembra "quasi" risolto.
            catchError((err: unknown) => {
              const message = err instanceof Error ? err.message : 'Errore
sconosciuto';

```

```

        return of<SearchResult>({ query: q, matches: [], error: message });
    }},
    ),
    ),
    tap(() => this.isSearching.set(false)),
    ),
    { initialValue: { query: '', matches: [] } as SearchResult },
);

private fakeSearch(query: string) {
    if (!query.trim()) {
        return of<SearchResult>({ query, matches: [] });
    }
    // Query magica per simulare un "server" che risponde con un errore
    // (es. 500, timeout, rete assente) – serve solo per la demo di
    // catchError qui sotto, mai fare query "magiche" così in un
    // servizio vero.
    if (query === 'error') {
        return throwError(() => new Error('Il server finto è caduto apposta'));
    }
    const matches = FAKE_REMOTE_TASKS.filter((t) =>
        t.toLowerCase().includes(query.toLowerCase()),
    );
    // Ritardo artificiale e VARIABILE in base alla lunghezza della query:
    // più corta = molto più lenta (query larghe = "il server finto" deve
    // scandagliare più risultati). Serve a rendere il race condition
    // riproducibile a comando invece che affidato al caso della rete
    // reale – utile per la demo, MAI da fare in un servizio vero.
    const artificialDelayMs = Math.max(250, 3000 - query.length * 1200);
    return of<SearchResult>({ query, matches }).pipe(delay(artificialDelayMs));
}
}

```

<src/app/features/task-search/task-search.html>

<h2 class="text-xl font-semibold text-slate-900">Cerca (demo RxJS)</h2>

<p class="mt-1 text-sm text-slate-500">

Prova a digitare velocemente "a" e poi subito "ab": la query più corta è volutamente più lenta di quella più lunga (vedi task-search.ts) – se `switchMap` funziona, il risultato finale corrisponde sempre all'ULTIMA query digitata, mai a una risposta lenta arrivata in ritardo.

</p>

<!-- [value] + (input) è un "two-way binding fatto a mano": leggiamo query() per mostrare il valore corrente, scriviamo query.set(...) a ogni digitazione. È l'equivalente esplicito di v-model in Vue, che sotto il cofano fa esattamente questa stessa coppia legame-in-lettura/legame-in-scrittura. Qui non usiamo [(ngModel)] (il "v-model" storico di Angular, richiede FormsModule) né Signal Forms (visto in task-form.ts) perché per un campo di ricerca senza validazione sarebbe complessità in più senza benefici: un signal semplice basta e avanza.

\$any(...) è un cast SOLO di template: \$event.target ha tipo generico EventTarget (potrebbe essere qualsiasi elemento del DOM), quindi TypeScript in modalità strictTemplates (vedi tsconfig.json) rifiuterebbe .value perché EventTarget non garantisce quella proprietà. \$any() dice "fidati, qui è un input" – stesso spirito di un cast "as any" nel .ts, ma con sintassi dedicata ai template. -->

<input
 type="text"
 [value]="query()"
 (input)="query.set(\$any(\$event.target).value)"
 placeholder="Cerca tra i task remoti finti..."
 class="mt-4 block w-full max-w-sm rounded-md border border-slate-300 px-3 py-2 text-sm shadow-sm focus:border-blue-500 focus:outline-none focus:ring-1 focus:ring-blue-500"
>

<!-- isSearching è un signal scritto dai due tap() nella pipe RxJS (task-search.ts): true dal momento in cui switchMap parte, false quando la ricerca (finta) risponde. Un effetto collaterale del flusso RxJS, letto qui come un signal qualsiasi – il template non sa né gli interessa che dietro ci sia un pipe RxJS. -->

@if (isSearching()) {
 <p class="mt-2 text-sm text-slate-400">Ricerca in corso...</p>
}

<!-- error è popolato solo dal catchError dentro switchMap (task-search.ts) – se catchError non ci fosse, questo ramo non servirebbe a nulla: l'intero stream morirebbe prima ancora di arrivare qui. -->

@if (results().error) {
 <p class="error mt-2 text-sm text-red-600">Errore: {{ results().error }}</p>
}

<!-- results() è il signal risultato finale del pipe (dopo toSignal()): un oggetto { query, matches }, non un semplice array – per questo leggiamo results().matches qui e results().query più sotto. Ogni lettura richiama il signal, ma nella stessa passata di change detection restituisce sempre lo stesso valore corrente (non rifà


```

        il lavoro del pipe RxJS, quello succede solo quando query() cambia
        davvero). track match: dato che i risultati sono semplici stringhe,
        tracciarle per il loro stesso valore basta – non serve un id
        dedicato come task.id altrove nel progetto. -->
<ul class="mt-4 max-w-sm divide-y divide-slate-200 rounded-lg border border-slate-
200 bg-white">
  @for (match of results().matches; track match) {
    <li class="px-4 py-2 text-sm text-slate-800">{{ match }}</li>
  } @empty {
    <li class="px-4 py-6 text-center text-sm text-slate-500">Nessun risultato</li>
  }
</ul>

<!-- data-testid: non è una feature di Angular, è pura convenzione HTML
(un attributo qualsiasi) usata qui apposta per rendere verificabile
da uno script esterno (il nostro script CDP di verifica) QUALE
query ha prodotto il risultato mostrato in un dato istante – è così
che abbiamo dimostrato che switchMap scarta le risposte tardive:
leggendo questo testo a tempi prestabiliti e controllando che non
"torni indietro" a una query precedente. -->
<p class="mt-2 text-xs text-slate-400" data-testid="answered-query">
  Risultato mostrato per la query: "{{ results().query }}"
</p>

```